

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина: Бэк-энд разработка**

**Отчет**

**Домашняя работа №2**

Выполнил:

Малахов Алексей

БР1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2025 г.

## Задача

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API

## Ход работы

Функциональные требования:

1. Аутентификация и авторизация: регистрация по email, вход с выдачей JWT-токенов, разграничение прав по ролям (арендатор, арендодатель).
2. Личный кабинет: редактирование профиля, список аренд, избранные объекты, история транзакций.
3. Каталог недвижимости: CRUD объектов, загрузка фотографий, поиск с фильтрацией по типу, цене, городу, страничная выдача.
4. Аренда: создание заявки, подтверждение/отклонение арендодателем, фиксация цены, сроков и залога, отслеживание статуса.
5. Платежи: регистрация оплаты, внесение и возврат залога, история операций.
6. Коммуникация: обмен сообщениями между пользователями в привязке к объекту, отметка о прочтении.
7. Отзывы: оценка и комментарий арендодателю после завершения аренды.

Нефункциональные требования:

1. Производительность: время ответа API не более 500 мс, индексы для оптимизации поиска.
2. Безопасность: хэширование паролей, валидация входных данных, HTTPS.
3. Надёжность: soft delete, логирование ошибок.
4. Поддерживаемость: документирование API через Swagger, единообразная структура ответов.

В качестве формата реализации API выбран подход Backend For Frontend. Данный выбор обусловлен сложностью спроектированной схемы базы данных (13 таблиц, 10 перечислений, 5 доменов), где большинство пользовательских экранов требуют агрегации данных из нескольких связанных таблиц. BFF позволяет инкапсулировать логику сборки данных на стороне сервера и предо-

ставлять клиенту готовые, оптимизированные под конкретный интерфейс ответы, сокращая количество запросов и упрощая фронтенд.

#### Эндпоинты API:

| №  | Метод  | Эндпоинт                        | Описание                         |
|----|--------|---------------------------------|----------------------------------|
| 1  | POST   | /auth/register                  | Регистрация нового пользователя  |
| 2  | POST   | /auth/login                     | Вход в систему                   |
| 3  | POST   | /auth/refresh                   | Обновление токена                |
| 4  | POST   | /auth/logout                    | Выход из системы                 |
| 5  | GET    | /profile                        | Получить профиль                 |
| 6  | PATCH  | /profile                        | Обновить профиль                 |
| 7  | GET    | /profile/dashboard              | Данные личного кабинета (BFF)    |
| 8  | POST   | /profile/roles                  | Добавить роль пользователю       |
| 9  | GET    | /properties                     | Поиск недвижимости с фильтрацией |
| 10 | POST   | /properties                     | Создать объект недвижимости      |
| 11 | GET    | /properties/{id}                | Полная страница объекта (BFF)    |
| 12 | PATCH  | /properties/{id}                | Обновить объект                  |
| 13 | DELETE | /properties/{id}                | Удалить объект (soft delete)     |
| 14 | POST   | /properties/{id}/photos         | Загрузить фото объекта           |
| 15 | DELETE | /properties/{pId}/photos/{phId} | Удалить фото                     |
| 16 | GET    | /favorites                      | Список избранных объектов        |
| 17 | POST   | /favorites/{propertyId}         | Добавить в избранное             |
| 18 | DELETE | /favorites/{propertyId}         | Удалить из избранного            |
| 19 | GET    | /rentals                        | Список аренд пользователя        |
| 20 | POST   | /rentals                        | Создать заявку на аренду         |
| 21 | GET    | /rentals/{id}                   | Детали аренды (BFF)              |
| 22 | PATCH  | /rentals/{id}/status            | Изменить статус аренды           |
| 23 | GET    | /transactions                   | История транзакций               |
| 24 | POST   | /transactions                   | Создать транзакцию               |
| 25 | GET    | /conversations                  | Список диалогов (BFF)            |
| 26 | POST   | /conversations                  | Начать новый диалог              |
| 27 | GET    | /conversations/{id}             | Диалог с сообщениями (BFF)       |
| 28 | POST   | /conversations/{id}/messages    | Отправить сообщение              |
| 29 | POST   | /conversations/{id}/read        | Отметить как прочитанные         |
| 30 | GET    | /landlords/{id}/reviews         | Отзывы об арендодателе (BFF)     |
| 31 | POST   | /reviews                        | Оставить отзыв                   |

## **Вывод**

В ходе работы сформулированы 7 функциональных и 4 нефункциональных требования к сервису аренды недвижимости. Выбран подход Backend For Frontend, обоснованный необходимостью агрегации данных из множества связанных таблиц. Спроектировано 31 эндпоинт в формате OpenAPI 3.0, охватывающий все модули системы: аутентификацию, профиль, каталог объектов, аренду, платежи, коммуникацию, отзывы и избранное. Для каждого эндпоинта описаны тела запросов, ответов и возможные ошибки.